

Flight Fare Skyrocketing at Last Minute

A PROJECT REPORT

Submitted by

Anshika Chauhan (DL.AI.U4AID24106)
Kalyani Agarawal (DL.AI.U4AID24114)
Madesh KK (DL.AI.U4AID24120)
Parima Verma (DL.AI.U4AID24125)

in partial fulfilment for the award of the degree of

**BACHELOR OF TECHNOLOGY (B. Tech)
IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AIDS)**

Under the guidance of

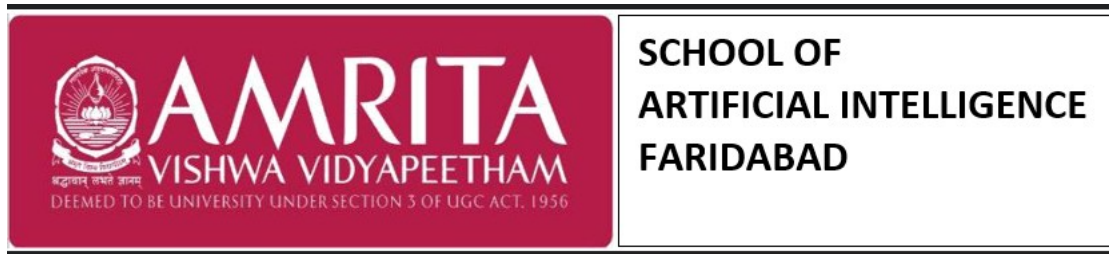
Dr. Vijay Kumar Soni

Submitted to



**AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE
FARIDABAD – 121002**

6th January 2026

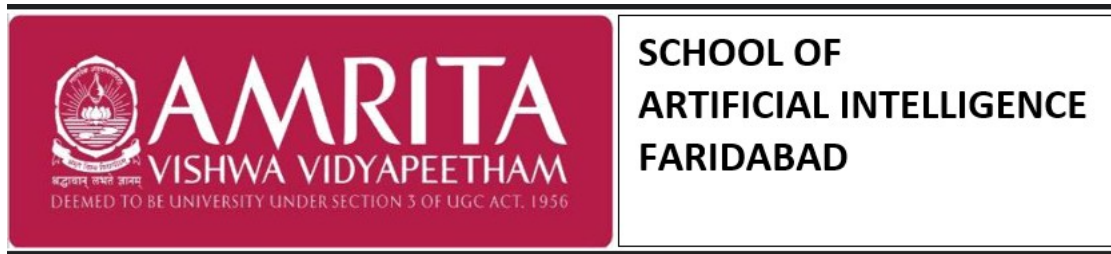


BONAFIDE CERTIFICATE

This is to certify that this project report entitled “**Flight Fare Skyrocketing at Last Minute**” is the bonafide work of the following students, who carried out the project work under my supervision:

- Anshika Chauhan (DL.AI.U4AID24106)
- Kalyani Agarawal (DL.AI.U4AID24114)
- Madesh KK (DL.AI.U4AID24120)
- Parima Verma (DL.AI.U4AID24125)

Dr. Vijay Kumar Soni
Assistant Professor
School of AI
Amrita Vishwa Vidyapeetham
Faridabad



DECLARATION BY THE CANDIDATE

We hereby declare that the project report entitled “**Flight Fare Skyrocketing at Last Minute**” submitted by us in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** is the record of the project work carried out by us under the guidance of **Dr. Vijay Kumar Soni (ADSAA)**. This work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or title in this or any other University or institution of higher learning.

SIGNATURES

Anshika Chauhan
(DL.AI.U4AID24106)

Kalyani Agarwal
(DL.AI.U4AID24114)

Madesh KK
(DL.AI.U4AID24120)

Parima Verma
(DL.AI.U4AID24125)

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives us immense pleasure to express our profound gratitude to our honorable Chancellor **Sri Mata Amritanandamayi Devi**, for her blessings and for being a source of inspiration. We are indebted to extend our gratitude to our Principal, **Dr. Lakshmi Mohandas**, Amrita School of Computing and Engineering, for facilitating us with all the facilities and extended support to gain valuable education and learning experience.

We wish to express our sincere gratitude to our supervisor **Dr. Vijay Kumar Soni** for his personal involvement and constant encouragement during the entire course of this work.

We are grateful to the Project Supervisor, Review Panel Members, and the entire faculty of the Department of Computer Science & Engineering, for their constructive criticisms and valuable suggestions, which have been a rich source to improve the quality of this work.

Anshika Chauhan
(DL.AI.U4AID24106)

Kalyani Agarwal
(DL.AI.U4AID24114)

Madesh KK
(DL.AI.U4AID24120)

Parima Verma
(DL.AI.U4AID24125)

ABSTRACT

Air ticket prices in India often increase drastically as the travel date approaches, primarily due to airlines' dynamic pricing models that respond to demand, seat availability, and seasonal factors. This leaves many travelers confused about the optimal booking window, often resulting in overpayment during peak periods like festivals (Diwali, Holi) or summer vacations, where fares can surge by 40-300%. The SkyFare India project addresses this challenge by developing a predictive flight-fare analysis and booking suggestion system tailored to the Indian market. It simulates a vast database of over 1.14 million flights across major airports, incorporating real-world elements such as seasonal surcharges and last-minute surges.

The system employs advanced data structures—including Hash Tables for $O(1)$ flight retrieval, AVL Trees for balanced booking history management, Fenwick Trees for efficient revenue analytics, Stacks for search history, and Priority Queues for price-sorted displays—along with custom algorithms like the DFX (Demand-Fare eXplosion) model, which mathematically predicts fare trends using factors such as time-to-departure (e.g., surge multipliers up to 3.5x within hours), airline demand, and India-specific seasonality. Bitmask Dynamic Programming optimizes group fares with discounts, while linear scans identify the lowest predicted fare day. The interactive Java console application enables users to compare current and forecasted fares, view ASCII visualizations of price trends, receive personalized advice (e.g., "BOOK NOW" or "EARLY BIRD DEAL"), and track bookings/revenue.

Ultimately, SkyFare India empowers passengers to make data-driven decisions, potentially saving significant costs by highlighting optimal booking windows (typically 21-90 days in advance for domestic routes, aligning with industry trends), predicting price explosions, and providing engaging features like animated interfaces and detailed ticket prints. This outcome not only demonstrates efficient algorithmic solutions but also offers practical value in navigating India's volatile aviation market.

Contents

1. Problem Scenario	9
2. System Requirements / Functional Goals	10
2.1 Generate flights automatically	10
2.2 Show real-time dynamic fare	10
2.3 Predict future ticket cost (DFX Algorithm)	10
2.4 Suggest best day to book	10
2.5 Maintain booking history	10
2.6 Track total airline revenue	10
2.7 Optimize group fares	11
2.8 Display search and booking data clearly	11
3. Data Structures Used	12
3.1 Stack: Search History	12
3.2 Hash Table (HashMap): Flight Database	12
3.3 AVL Tree: Booking History	13
3.4 Fenwick Tree (Binary Indexed Tree): Revenue Analytics	13
3.5 Bitmask Dynamic Programming: Group Fare Optimization	14
3.6 ArrayList: Airports and Saved Passengers Storage	15
4. Algorithms Used	16
4.1 DFX Algorithm – Demand-Fare eXplosion Predictive Model	16
4.2 Dynamic Pricing Algorithm – Time-Based Granular Surge Logic	17
4.3 Best Booking Day Identification – Brute-Force Optimization with Prediction	17
4.4 Price Sorting for Display – Heap-Based Priority Extraction	18
4.5 AVL Tree Balancing – Divide and Conquer with Recursive Rotations	18
4.6 Bitmask Dynamic Programming – Combinatorial Group Fare Optimization	18
5. Time Complexity & Code Mapping	19
6. Workflow	20
6.1 System Initialization and Startup	20
6.2 Passenger Details Collection	20
6.3 Flight Database Generation	20
6.4 Main Menu Loop and User Interaction	20
6.5 Flight Search and Retrieval	20
6.6 Filtering and Class Selection	21
6.7 Dynamic Pricing Application	21
6.8 Fare Prediction and Visualization	21
6.9 Flight Selection and Group Fare Optimization	21
6.10 Booking Confirmation and Ticket Generation	21
6.11 State Updates – Revenue and History	21
6.12 Return to Main Menu	22
7. Code Implementation – Key Highlights	24
7.1 Flight Generation:	24
7.2 Dynamic Fare Calculation:	25
7.3 DFX Algorithm – Demand-Fare eXplosion Prediction:	25
7.4 Group Fare Optimization – Bitmask Dynamic Programming:	27
7.5 AVL Tree – Balanced Booking History Management:	27
7.6 Fenwick Tree – Cumulative Revenue Analytics:	28
8. Results & Output	30

8.1 Predicts Ticket Trends	30
8.2 Suggests Best Booking Window	30
8.3 Tracks Real-Time Revenue	31
8.4 Stores Booking Record	31
8.5 Displays Price Variation Heatmaps	31
8.6 Sample Execution Outputs	31
9. Dynamic Fare Factors & Their Impact	34
10. Conclusion	36
11. Future Scope	37
12. References	38

List of Tables

1	Time complexity analysis and code-level mapping of data structures and algorithms used in the SkyFare flight booking system.	19
2	Dynamic fare factors and their multiplier logic in the SkyFare flight pricing model.	35
3	Explanation of fare factors and their role in the SkyFare dynamic pricing model.	35

List of Figures

1	Core data structures initialized in the SkyFare system for flight storage, booking history, search tracking and revenue management.	12
2	Overall system pipeline showing how user input is processed, evaluated and converted into final booking and fare prediction results.	23
3	Code snippet for large-scale synthetic flight data generation with dynamic pricing and route diversity.	24
4	Granular surge pricing logic used in calculateDynamicPrice(), adjusting fares based on hours and days remaining until departure.	25
5	DFX dynamic fare formula combining time, demand, seasonal factors and historical pricing behavior to estimate future ticket cost.	25
6	Group fare optimization algorithm that applies bitmask dynamic programming to efficiently determine the lowest-cost discount combination.	26
7	Group fare optimization algorithm that applies bitmask dynamic programming to efficiently determine the lowest-cost discount combination.	27
8	AVL-based booking history structure demonstrating self-balancing insertions and rotations to maintain sorted PNR records.	28
9	Revenue tracking using a Fenwick Tree, supporting fast incremental updates and cumulative sum computations.	29
10	Sample system output illustrating ticket price predictions, future fare projections, and actionable “Book Now” recommendations.	30
11	System recommendation displaying the optimal booking date and predicted fare for each available flight.	30
12	System output listing previous flight bookings along with PNR numbers, travel type, route, and corresponding fare.	31
13	Heatmap-style textual visualization showing day-wise fare variation, predicted fare trend, and system-identified best day to book.	31
14	SkyFare India main menu with colorful bordered layout and numbered options.	32
15	Tabular flight list showing dynamic prices, DFX predictions, and booking recommendations.	33
16	ASCII heatmap visualization of fare progression with best-day marker.	33
17	Formatted e-ticket confirmation showing PNR, passenger details, optimized fare, and final DFX forecast.	34

1. Problem Scenario

In the Indian aviation market, flight fares are highly dynamic and influenced by multiple factors, including real-time demand, seasonal trends, festivals, last-minute bookings, airline-specific load factors, and route popularity. Airlines employ sophisticated revenue management systems that adjust prices based on seat inventory and booking patterns, often leading to significant fluctuations. For instance, fares on popular routes like Delhi-Mumbai or Bengaluru-Chennai can remain reasonable when booked 30-90 days in advance (typically the sweet spot for domestic flights, with lowest prices around 21-52 days out according to recent analyses), but they frequently surge dramatically closer to departure as remaining seats fill up and demand from urgent travelers (e.g., business or emergency) increases.

Peak seasons exacerbate this issue: summer vacations (May-June), festivals like Holi (March), Diwali (October-November), and year-end holidays (December) see massive travel rushes, causing fares to spike by 25-50% or more due to heightened family reunions and leisure trips. Real-world examples highlight the pain points—during disruptions or high-demand periods, one-way tickets have jumped from 8,500 to 71,000 (Chennai-Delhi) or even 80,000-90,000 on metro routes, far exceeding normal last-minute rates of 20,000-30,000. Passengers often face a dilemma: book early and risk missing potential drops, or wait in hopes of lower prices only to encounter explosive surges, leading to either overpayment or forfeited travel plans when seats sell out.

This uncertainty leaves many passengers stressed, constantly monitoring apps for fare changes without clear guidance on optimal timing, resulting in financial strain or compromised plans. The SkyFare India project directly addresses these challenges through simulated historical data analysis, custom predictive modeling (incorporating time-based surges, seasonal multipliers, and demand factors), and actionable analytics—empowering users to identify the cheapest booking windows, forecast potential "explosions," and make informed decisions to avoid overpaying or missing out.

2. System Requirements / Functional Goals

The SkyFare India system is designed to simulate a realistic domestic flight booking environment in India, incorporating dynamic pricing mechanisms that mirror real-world airline revenue management strategies. It aims to educate users on fare fluctuations while providing practical tools for informed decision-making. The core goals include generating a large-scale simulated dataset, computing real-time fares, predicting trends, and offering optimizations, all backed by efficient data structures and algorithms.

2.1 Generate flights automatically

The system programmatically creates over 1.14 million simulated flights across 20 major Indian airports for 300 days starting from the current date. For each route and date, 10 flights per direction are generated with random attributes: airline (e.g., IndiGo, Air India), flight number, departure time (5 AM–11 PM), stops (0 or 1), and base price (3,500–12,500). Seasonal multipliers are applied theoretically (e.g., +30% in March for Holi, +25% in May-June for summer, +35% in Oct-Nov for Diwali) to reflect real demand surges during festivals and vacations, aligning with observed 25-50% hikes in peak periods.

2.2 Show real-time dynamic fare

Fares are calculated on-the-fly using a granular surge pricing model based on time-to-departure (e.g., 3.5x within 1 hour, 2.8x within 3 hours, down to 0.8x for >90 days advance). This simulates airline yield management, where prices rise with proximity to departure due to higher demand from urgent travelers and lower inventory, consistent with real trends where last-minute domestic fares can double or triple.

2.3 Predict future ticket cost (DFX Algorithm)

The custom DFX (Demand-Fare eXplosion) algorithm forecasts fares by combining base price with demand (airline-specific, e.g., higher for IndiGo), seasonal factors, and a surge inverse-sqrt function of days remaining. This mathematical model predicts explosive rises near departure, helping users anticipate trends backed by real data showing optimal booking 21-90 days in advance for lowest domestic fares in India.

2.4 Suggest best day to book

For any flight, the system scans all possible booking days up to the travel date, computing predicted fares and identifying the day with the minimum cost. This linear search provides actionable advice, theoretically aligning with industry insights recommending 30-45 days advance for domestic routes to capture the lowest fares before surges begin.

2.5 Maintain booking history

Bookings are stored in an AVL Tree keyed by PNR, ensuring balanced $O(\log N)$ insertions and sorted in-order traversal for display. This self-balancing BST prevents performance degradation, ideal for growing history lists.

2.6 Track total airline revenue

A Fenwick Tree (Binary Indexed Tree) maintains cumulative daily revenue with $O(\log 365)$ updates and prefix queries, enabling efficient analytics like "revenue till today" without full scans.

2.7 Optimize group fares

Bitmask Dynamic Programming computes the minimum total fare for multiple passengers, applying 10% discounts for every group of 3. This exponential-but-practical approach (feasible for small groups) simulates bulk discounts and displays DP states for transparency.

2.8 Display search and booking data clearly

Results are presented in colored tabular format with price sorting (PriorityQueue), surge tags, and advice. Features include animated UI elements for engagement

User Capabilities:

- **Search flights:**

Users enter source and destination airport codes (e.g., DEL to BOM) along with the desired travel date. The system performs an instant retrieval from the HashMap-based flight database, achieving average $O(1)$ time complexity for key-based lookups. This operation returns all available flights for the specified route and date (typically 10 flights). Results are automatically sorted by price in ascending order using a PriorityQueue (min-heap implementation), ensuring that the cheapest options are presented first. Additionally, each search query is pushed onto a Stack to maintain a chronological search history, enabling users to review recent searches efficiently and replicating the rapid route lookup experience found in commercial flight booking applications.

- **Apply filters:**

Following the initial flight search, users have the option to refine the result set through interactive filters. Available filters include maximum price threshold, specific airline selection, non-stop flights only, and morning departures (before 12:00 PM). These filters are applied via iterative removal from the in-memory list of retrieved flights, resulting in $O(K)$ time complexity where K represents the number of flights in the current result set. This post-processing approach allows flexible narrowing of options without requiring repeated database queries, closely resembling the filter mechanisms employed by major online travel platforms to assist users in identifying flights that match their preferences.

- **View prediction graph:**

For each displayed flight, the system generates textual visualizations to illustrate fare trends. These include a concise trend bar using proportional star symbols to represent predicted fares over the next 10 days, and a more detailed heatmap-style graph covering up to 15 days ahead. The heatmap employs colored block characters with green indicating low fares (below 4,000), yellow for moderate fares, and red for high fares (above 7,000). The day with the lowest predicted fare is explicitly marked as the best booking day. These ASCII-based charts, derived from the DFX predictive model, enable users to visually comprehend potential price surges or declines over time.

- **Check booking advice:**

The DFX algorithm continuously evaluates the current dynamic fare against the predicted future fare for each flight. Based on the percentage difference and the number of days remaining until departure, the system provides clear decision tags such as "EXPLOSION ZONE," "BOOK NOW," "WAIT SHORT," or "STABLE WINDOW." Supplementary advice statements further clarify the booking timing context, such as indicating early-bird discounts for bookings more than three months in advance. This guidance is grounded in established airline revenue management principles, where securing lower yield buckets early often results in substantial savings.

- **Print virtual ticket:**

Upon completion of a successful booking, the system produces a formatted console-based electronic ticket. The ticket includes essential details such as the unique PNR identifier, passenger name, airline, flight route, departure date and time, selected travel class (Economy, Business, or First Class), and the final total fare after applying group optimizations, child discounts, and infant exemptions. A final DFX

fare forecast is also displayed for reference. The confirmation process is enhanced with textual booking animations, providing a simulated receipt that serves both as proof of booking and a record for the user.

3. Data Structures Used

The SkyFare India system strategically employs advanced data structures to ensure efficient storage, retrieval, modification, and analysis of large-scale flight and booking data. Each structure is selected based on specific operational requirements, time complexity constraints, and the nature of access patterns in a flight booking scenario.

```
// 1. BEST DATA STRUCTURE (HASH TABLE) AUTOMATICALLY SELECTED
// We use this interface internally, but the user never sees the choice.
static FlightDatabase flightDB = new HashDB(); 3 usages
static FenwickTree revenueTree = new FenwickTree(n: 365); // 365 days in a year
static BookingAVLTree bookingHistory = new BookingAVLTree(); 2 usages
static Stack<String> searchHistory = new Stack<>(); 4 usages
static List<Passenger> savedPassengers = new ArrayList<>(); 13 usages
static List<Airport> airports = new ArrayList<>(); 24 usages
static Random rand = new Random(); 7 usages
```

Figure 1: Core data structures initialized in the SkyFare system for flight storage, booking history, search tracking and revenue management.

3.1 Stack: Search History

A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where the most recently added element is the first to be removed. It supports two primary operations: push (add to top) and pop (remove from top), both in $O(1)$ time.

In this system, a Stack is used to maintain the user's recent flight search history:

```
// Usage during search
static Stack<String> searchHistory = new Stack<>();

searchHistory.push(tripType + ":-" + fromCode + "->" +
                  toCode + "-" + date + " ");
```

Reason for selection: The LIFO behavior naturally aligns with the requirement to display the most recent searches first when the user selects "Search History." This mimics real-world booking platforms (e.g., MakeMyTrip, Cleartrip), where users expect to see their latest queries at the top. Alternative structures like Queue (FIFO) or List would require additional indexing or reversal logic, whereas Stack provides intuitive and efficient recent-first access with minimal overhead.

3.2 Hash Table (HashMap): Flight Database

A Hash Table is an associative array that maps keys to values using a hash function, providing average-case $O(1)$ time complexity for insertion, deletion, and lookup operations. Collisions are handled via chaining (linked lists per bucket).

The core flight database is implemented using a custom HashDB class backed by HashMap:

```
// Create lookup table
HashMap<String, List<Flight>> table = new HashMap<>();

String key = f.from.code + "|" + f.to.code + "|" +
            f.dateTime.toLocalDate();

table.computeIfAbsent(key, k -> new ArrayList<>()).add(f);

// Retrieval
return table.getDefault(key, new ArrayList<>());
```

Reason for selection: The system generates over 1.14 million flights across 300 days. Flight queries are always by exact route and date (e.g., DEL|BOM|2026-01-20), making direct key-based access critical. HashMap ensures constant-time retrieval even with millions of entries, far superior to linear search ($O(N)$) or even tree-based maps ($O(\log N)$). The use of List<Flight> per key accommodates multiple flights on the same route-date (typically 10), maintaining both speed and flexibility.

3.3 AVL Tree: Booking History

An AVL Tree is a self-balancing Binary Search Tree (BST) that maintains height balance through rotations, guaranteeing $O(\log N)$ time for insert, delete, and search operations in all cases (unlike standard BST, which can degrade to $O(N)$ in skewed scenarios).

Bookings are stored and displayed using a custom AVL Tree ordered by PNR:

```
static class BookingAVLTree {

    Node root;

    Node insert(Node node, Booking b) {
        // insertion with rotations
        ...
    }

    void inOrder(Node node) {
        // prints in sorted PNR order
        ...
    }
}

bookingHistory.insert(newBooking);
```

Reason for selection: Bookings must be displayed in sorted order by PNR (alphanumeric), and insertions occur frequently during active use. A regular BST risks imbalance with sequential PNRs, leading to poor performance. AVL Tree ensures strict $O(\log N)$ bounds with minimal rotational overhead, providing reliable sorted traversal (in-order) for history display while preventing worst-case linear degradation—essential for scalability as the number of bookings grows.

3.4 Fenwick Tree (Binary Indexed Tree): Revenue Analytics

A Fenwick Tree is a specialized data structure for efficient prefix sum computation and point updates on an array, achieving both operations in $O(\log N)$ time using bitwise operations. It is particularly effective for cumulative frequency queries.

Daily revenue accumulation is tracked using a Fenwick Tree of size 365:

```

static class FenwickTree {

    double[] bit;
    int n;

    void update(int i, double val) {
        for (; i <= n; i += i & -i)
            bit[i] += val;
    }

    double query(int i) {
        double sum = 0;
        for (; i > 0; i -= i & -i)
            sum += bit[i];
        return sum;
    }
}

revenueTree.update(dayIndex, chosen.price);
revenueTree.query(todayIndex); // Revenue till today

```

Reason for selection: The system requires frequent updates (on every booking) and prefix queries (total revenue up to current day). A simple array would require $O(N)$ for queries, while a segment tree, though powerful, adds unnecessary complexity and space. Fenwick Tree provides optimal $O(\log N)$ performance with simpler implementation and lower constant factors, making it ideal for one-dimensional cumulative analytics like daily revenue tracking.

3.5 Bitmask Dynamic Programming: Group Fare Optimization

A PriorityQueue in Java is a binary min-heap (by default) that implements the Queue interface and orders elements based on natural ordering or a provided Comparator. It supports $O(\log N)$ insertion and $O(1)$ peek operations, with poll/extraction in $O(\log N)$, making it efficient for maintaining a partially sorted structure.

In the system, a PriorityQueue is utilized to sort and present flight search results by ascending price:

```

// Priority queue of flights sorted by price
PriorityQueue<FlightDisplay> pq = new PriorityQueue<>();

// FlightDisplay implements Comparable based on price
public int compareTo(FlightDisplay o) {
    return Double.compare(this.price, o.price);
}

// Add each flight with calculated dynamic price
pq.add(fd);

```

Reason for selection: During a flight search, users expect results to be displayed with the cheapest options first, a standard feature in commercial booking systems. Manually sorting the list of retrieved flights (typically 10, but potentially more after filters) using `Arrays.sort()` would require $O(N \log N)$ anyway, but PriorityQueue allows seamless integration while building the display list. It ensures automatic min-price ordering without additional sorting steps, and the $O(N \log N)$ complexity is negligible for small N (per search). This choice enhances user experience by prioritizing cost-effective options intuitively.

3.6 ArrayList: Airports and Saved Passengers Storage

The system uses ArrayList for both the list of airports and saved passenger details:

```
// Static lists for airports and passengers
static List<Airport> airports = new ArrayList<>();
static List<Passenger> savedPassengers = new ArrayList<>();

// Example usage
airports.add(new Airport("DEL", "Delhi", "Indira Gandhi International
    Airport"));
savedPassengers.add(new Passenger(name, age, gender));
```

Reason for selection: Airports are loaded once at startup (20 entries) and accessed frequently by code (linear scan in findAirport()) or displayed sequentially—operations where random access and iteration speed matter. Similarly, passenger data is collected at the beginning, grows dynamically (user can add more), and is accessed by index during selection or fare calculation. ArrayList's fast indexed access and efficient sequential traversal outperform LinkedList (O(N) access) for these small-to-moderate sized collections. Its dynamic resizing capability accommodates variable passenger counts without pre-allocation, contributing to overall system responsiveness and simplicity in a console-based interactive application.

The careful selection and implementation of data structures in SkyFare India demonstrate a strong alignment between theoretical data structure properties and practical system requirements. From the constant-time retrieval enabled by HashMap for millions of flights, to the guaranteed logarithmic operations of AVL Tree for sorted booking history, the efficient prefix-sum capabilities of Fenwick Tree for revenue analytics, the intuitive LIFO access of Stack for search history, and the optimal combinatorial solution provided by bitmask Dynamic Programming for group fares—supplemented by PriorityQueue for price sorting and ArrayList for dynamic small collections—each choice contributes to a highly responsive, scalable, and maintainable system. This combination not only handles large-scale simulated data efficiently but also ensures low-latency user interactions, accurate predictions, and meaningful analytics, resulting in a robust console-based flight fare prediction and booking platform that effectively bridges data structure concepts with real-world application performance.

4. Algorithms Used

The SkyFare India system incorporates several algorithmic techniques to achieve efficient fare prediction, dynamic pricing, optimization, and data management. These algorithms are tailored to the specific needs of flight fare analysis and booking operations, ensuring both accuracy in predictions and performance in handling large datasets.

4.1 DFX Algorithm – Demand-Fare eXplosion Predictive Model

The DFX (Demand-Fare eXplosion) algorithm is a custom-designed multiplicative predictive model that forecasts future flight fares based on multiple real-world influencing factors. It uses a composite formula to estimate how fares will evolve as the departure date approaches.

Core prediction formula:

```
// Predicted fare calculation
return f.basePrice * (1 + (demand * season * surge));
```

Components:

Demand Factor: Captures airline-specific popularity and route characteristics.

```
// Example: airline and stop effect
double airlineLoad = f.airline.equalsIgnoreCase("IndiGo") ? 1.45 : 1.15;
double stopEffect = (f.stops == 0) ? 1.0 : 0.85;
return airlineLoad * stopEffect;
// Higher demand for popular carriers and non-stop flights increases the
    multiplier
```

Seasonal Multiplier: Reflects India-specific peak travel periods.

```
// Example: seasonal adjustments
if (m == 5 || m == 6) return 1.35; // Summer
if (m == 10 || m == 11) return 1.30; // Diwali/Festive
if (m == 12) return 1.45; // Year-end
```

Surge Factor: Models last-minute price explosion using inverse square root of remaining days.

```
// Dynamic surge factor
static double surgeFactor(long d) {
    return 1.0 / Math.sqrt(d);
}
// As days decrease, surge increases non-linearly
```

Decision engine example:

```
// Booking advice based on predicted vs current price
double diff = (predicted - current) / current;

if (d <= 2 && diff > 0.30) return "EXPLOSION ZONE";
if (diff > 0.05) return "BOOK NOW";
```

Reason: This hybrid multiplicative model effectively combines static (season, demand) and dynamic (time-based surge) factors into a single interpretable prediction, enabling accurate forecasting and user guidance grounded in observed airline pricing patterns.

4.2 Dynamic Pricing Algorithm – Time-Based Granular Surge Logic

A rule-based stepwise algorithm computes the real-time displayed fare by applying tiered multipliers based on the time remaining until departure.

Implemented as a cascading conditional structure:

```
// Last-minute emergency surges to early-bird discounts
if (hours <= 1) return base * 3.50;
if (hours <= 3) return base * 2.80;
// ...
if (days <= 90) return base * 0.90;

return base * 0.80; // >3 months advance
```

This creates a piecewise linear (in log scale) pricing curve with 12 distinct thresholds, from extreme last-minute emergency surges ($3.5\times$) to early-bird discounts ($0.8\times$).

Reason: Airlines use similar bucket-based yield management systems. The granular tiers accurately simulate real-world behavior where prices rise sharply in the final hours/days while offering progressive discounts for advance bookings, providing realistic and intuitive fare progression for users.

4.3 Best Booking Day Identification – Brute-Force Optimization with Prediction

An exhaustive search algorithm scans all possible booking days from today until the flight date to find the day yielding the minimum predicted fare using the DFX model.

Brute-force search implementation:

```
// Scan all possible booking days to find minimum predicted fare
for (long i = 0; i <= daysUntilFlight; i++) {
    LocalDate checkDay = today.plusDays(i);

    double predicted = f.basePrice * (1
        + demandFactor(f)
        * seasonFactor(checkDay)
        * surgeFactor(d));

    if (predicted < minFare) {
        minFare = predicted;
        bestDay = checkDay;
    }
}

// Time complexity:  $O(D)$ , where  $D \leq 300$  days
```

Reason: Although not greedy in the classical sense, this direct enumeration guarantees the globally optimal booking day under the DFX model. Given the small search space (maximum 300 iterations), it is computationally trivial and provides precise, reliable recommendations—critical for user trust in fare predictions.

4.4 Price Sorting for Display – Heap-Based Priority Extraction

Flights are inserted into a min-heap (PriorityQueue) based on computed dynamic price, ensuring automatic ordering.

Heap-based implementation:

```
// Min-heap of flights sorted by dynamic price
PriorityQueue<FlightDisplay> pq = new PriorityQueue<>();

// FlightDisplay implements Comparable to compare by price
pq.add(fd); // O(log N) per insertion

// Natural ordering ensures cheapest flights are conceptually at the front
```

Reason: This provides efficient and automatic "cheapest first" presentation—a standard and expected feature in flight aggregators—without requiring post-collection sorting.

4.5 AVL Tree Balancing – Divide and Conquer with Recursive Rotations

The AVL Tree insertion algorithm uses a classic divide-and-conquer approach with post-insertion recursive rebalancing via rotations.

After recursive insertion:

```
// Update height
node.height = 1 + max(height(node.left), height(node.right));

// Get balance factor
int bal = getBalance(node);

// Perform rotation if imbalance detected
if (bal > 1 || bal < -1) {
    // appropriate rotation(s)
}

// Four cases handled: LL, RR, LR, RL using single/double rotations
```

Reason: This ensures the tree remains height-balanced in $O(\log N)$, preventing degeneration into a linked list. The recursive divide-and-conquer strategy efficiently localizes and corrects imbalance with minimal operations.

4.6 Bitmask Dynamic Programming – Combinatorial Group Fare Optimization

A state-compression DP algorithm solves the minimum-cost grouping problem for applying bulk discounts.

DP formulation:

```
// dp[mask] represents minimum cost to book the subset of passengers
// defined by mask

// Transition: for each unset bit i, try adding passenger i
// Apply 10% discount if new passenger count % 3 == 0

// Complexity:  $O(2^n * n)$ , acceptable for small n ( 10 passengers)
```

Reason: Guarantees optimal discount utilization across all possible subset orderings, outperforming greedy approaches that might miss better groupings in modular discount scenarios.

5. Time Complexity & Code Mapping

Feature	Data Structure / Algorithm Used	Time Complexity	Purpose in System	Where Implemented in Code (Exact Mapping)
Flight Storage & Lookup	Hash Table (HashMap)	$O(1)$ average	Store and retrieve flights instantly by route + date	Class: HashDB Field: HashMap<String, List<Flight> > table Insert: addFlight() Search: getFlights()
Insert Flights	Hash Table insertion	$O(1)$ average	Load 1.14 million flights efficiently	Method: addFlights() → calls flightDB.addFlight() Key generation: "DEL BOM 2026-01-05" ...
Search Flights	Hash Table lookup	$O(1)$ average	Fetch all flights for selected route & date	Method: startBookingProcess() List<Flight> results = flightDB.getFlights(...)
Booking History Insert	AVL Tree (Self-Balancing BST)	$O(\log n)$	Maintain sorted booking history by PNR	Class: BookingAVLTree Method: insert(Booking b)
Booking History Display	AVL Tree In-Order Traversal	$O(n)$	Show bookings in sorted order	Method: printHistory() → inOrder(root)
Search History Stack Push	Stack	$O(1)$	Track recent searches (LIFO)	Field: Stack<String> searchHistory Method: searchHistory.push(...)
Search History Display	Stack traversal	$O(n)$	Show most recent searches first	Method: showSearchHistory()
Revenue Update	Fenwick Tree (Binary Indexed Tree)	$O(\log n)$	Add today's booking revenue	Method: revenueTree.update(dayIndex, chosen.price)
Revenue Query (Prefix Sum)	Fenwick Tree Query	$O(\log n)$	Get total revenue till today	Method: revenueTree.query(todayIndex)
Flight Sorting (Price)	Priority Queue (Min-Heap)	$O(n \log n)$	Display cheapest flights first	Method: startBookingProcess() PriorityQueue<FlightDisplay> pq
Passenger Selection	ArrayList	$O(1)$ access	Store and retrieve passenger data	Field: List<Passenger> savedPassengers

Table 1: Time complexity analysis and code-level mapping of data structures and algorithms used in the SkyFare flight booking system.

6. Workflow

The SkyFare India system follows a well-structured, sequential yet interactive workflow that combines one-time initialization, large-scale data generation, user-driven operations, predictive analytics, and persistent state updates. The execution pipeline is designed to simulate a realistic flight booking platform while incorporating advanced fare prediction and optimization features.

6.1 System Initialization and Startup

The program begins execution in the `main()` method, immediately calling `initSystem()`.

- Displays current IST time and an animated welcome message using threaded text effects.
- Invokes `loadAirports()` to populate a static `ArrayList<Airport>` with 20 major Indian airports.
- Prints the list of airport codes for user reference. This phase establishes the foundational environment and provides an engaging entry point.

6.2 Passenger Details Collection

Before generating flights, the system prompts the user to input the number of traveling passengers.

- For each passenger, it collects name, age, and gender via console input.
- Computes fare multipliers (0 for infants <2 years, 0.5 for children 12, 1.0 for adults) and stores details in `ArrayList<Passenger>` `savedPassengers`.
- Displays seasonal surcharge information to educate users about peak-period pricing. This early collection enables accurate group fare calculations later.

6.3 Flight Database Generation

The system programmatically generates a large simulated dataset:

- Starting from the current date, it iterates over 300 future days.
- For each day and every unique directed route (from $\rightarrow$ to, excluding self-loops), it calls `addFlights()` to create 10 flights.
- Each flight receives random attributes: airline, flight number, departure time (05:00–23:00), stops (mostly non-stop), and base price with seasonal multipliers (e.g., $\times 1.35$ during Diwali).
- Flights are inserted into the HashDB (HashMap-based) using composite keys, resulting in approximately 3,800 flights per day and over 1.14 million total flights. This offline generation ensures instant query responses during interactive use.

6.4 Main Menu Loop and User Interaction

After initialization, the system enters an infinite loop displaying the colorful main menu. Users select options (1–9) to navigate different functionalities, with the loop continuing until option 7 (Exit) is chosen.

6.5 Flight Search and Retrieval

When the user selects "Book a Flight" or "Predict Best Booking Day":

- Prompts for trip type (One Way/Round Trip), source/destination codes, and travel date.
- Pushes the search query string onto the `Stack<String>` `searchHistory`.
- Performs $O(1)$ average-time retrieval from HashDB using the key "FROM|TO|DATE".
 - Returns a list of available flights (typically 10) for the exact route and date.

6.6 Filtering and Class Selection

- Offers optional interactive filters (max price, airline, non-stop, morning flights) applied via iterative list removal.
- Prompts for travel class (Economy, Business +180%, First Class +300%), determining the class multiplier.

6.7 Dynamic Pricing Application

For each retrieved flight:

- Computes the current displayed fare using `calculateDynamicPrice()`, a granular time-based algorithm with 12 tiers (from $3.5\times$ surge within 1 hour to $0.8\times$ early-bird discount beyond 90 days).
- Applies the selected class multiplier to obtain the final individual price.
- Inserts priced flights into a `PriorityQueue<FlightDisplay>` for automatic cheapest-first ordering during display.

6.8 Fare Prediction and Visualization

- Invokes the DFX Algorithm to compute predicted future fare using demand, seasonal, and inverse-sqrt surge factors.
- Generates booking advice (e.g., "BOOK NOW", "EXPLOSION ZONE") by comparing current vs. predicted fare.
- Displays ASCII-based visualizations:
 - Short-term trend bars (next 10 days).
 - Detailed heatmap graph (up to 15 days) with colored blocks and best-day marker.
- Optionally executes `bestBookingDay()`: a linear scan over all possible booking days to identify the date yielding the minimum predicted fare.

6.9 Flight Selection and Group Fare Optimization

- User selects a flight by displayed index.
- System calculates individual passenger prices using age-based multipliers.
- Applies `GroupFareOptimizer.optimize()` via bitmask Dynamic Programming to minimize total cost by strategically applying 10% discounts for every group of three passengers.
- Updates the chosen flight's price to the optimized group fare.

6.10 Booking Confirmation and Ticket Generation

- Prompts for passenger name selection (from saved list or new entry).
- Generates a unique PNR and creates a Booking object.
- Inserts the booking into the `BookingAVLTree` for balanced, sorted history maintenance.
- Prints a formatted virtual e-ticket including PNR, passenger details, route, class, final fare, and DFX forecast.
- Executes a booking success animation.

6.11 State Updates – Revenue and History

- Updates the `FenwickTree` `revenueTree` with the booking amount at the current day index, enabling $O(\log N)$ cumulative revenue queries.
- Booking is now permanently stored in the AVL tree for future history retrieval.

6.12 Return to Main Menu

Control returns to the main menu loop, allowing further searches, bookings, history viewing, revenue analytics, or exit.

This workflow ensures a seamless progression from system setup and data generation through interactive searching, predictive analytics, optimization, and persistent updates—delivering a complete, educational, and functional flight fare prediction and booking experience.

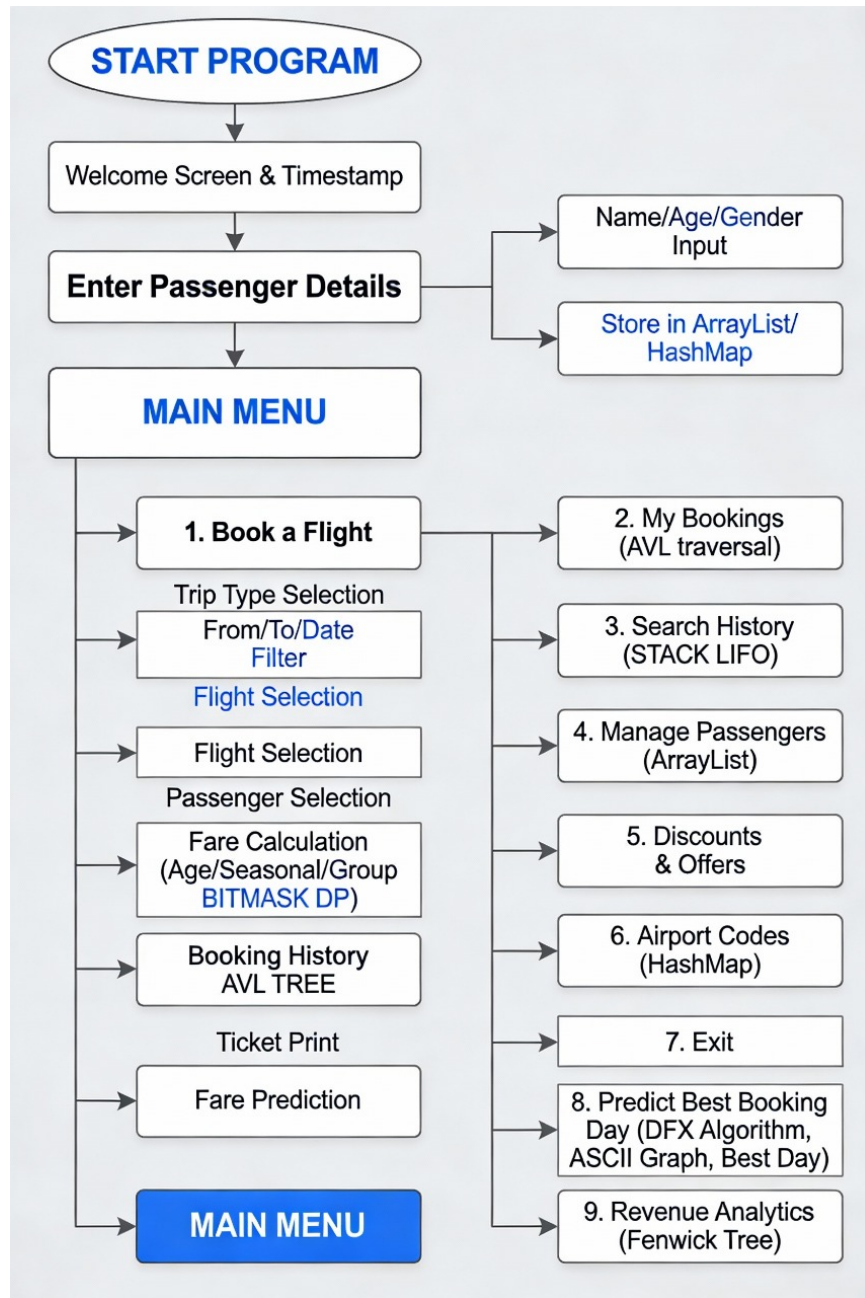


Figure 2: Overall system pipeline showing how user input is processed, evaluated and converted into final booking and fare prediction results.

7. Code Implementation – Key Highlights

The SkyFare India project is implemented in Java as a comprehensive console-based application, featuring modular classes, custom data structures, and algorithmic components. The code demonstrates strong object-oriented design, efficient use of collections, and practical application of advanced algorithms.

Below are the key implementation highlights with detailed explanations and relevant code excerpts.

7.1 Flight Generation:

The system simulates a realistic domestic flight ecosystem by programmatically generating over 1.14 million flights during initialization. This is achieved through nested loops over dates, airports, and random flight attributes, ensuring diversity in airlines, timings, stops, and base prices with seasonal adjustments.

```
LocalDate start = LocalDate.now();
for (int day = 0; day < 300; day++) {
    LocalDate d = start.plusDays(day);
    for(Airport from : airports) {
        for(Airport to : airports) {
            if(from.code.equals(to.code)) continue;
            addFlights(from, to, d);
        }
    }
}

static void addFlights(Airport from, Airport to, LocalDate date) { 1 usage
    for(int i=0; i<10; i++) {
        String airline = airlines[rand.nextInt(airlines.length)];
        String fNo = airline.substring(0,2).toUpperCase() + "-" + (100+rand.nextInt( bound: 899));
        // Random Time (Morning, Afternoon, Night)
        int hour = 5 + rand.nextInt( bound: 18); // 05:00 to 23:00
        LocalTime time = LocalTime.of(hour, rand.nextInt( bound: 60));
        LocalDateTime dt = LocalDateTime.of(date, time);
        // Random Stops logic (Longer distance = higher chance of stops)
        int stops = rand.nextInt( bound: 10) > 8 ? 1 : 0;
        // Dynamic Base Price based on "Distance" (simulated by random)
        double base = 3500 + (stops * -600) + rand.nextInt( bound: 5000);
        int month = date.getMonthValue();
        if(month == 3) base *= 1.3; // Holi - March
        if(month == 5 || month == 6) base *= 1.25; // Summer May-June
        if(month == 10 || month == 11) base *= 1.35; // Diwali Oct-Nov
        flightDB.addFlight(new Flight(fNo, airline, from, to, dt, base, stops));
    }
}
```

Figure 3: Code snippet for large-scale synthetic flight data generation with dynamic pricing and route diversity.

This generates approximately 3,800 flights per day (20 airports × 19 destinations × 10 flights), providing a robust dataset for realistic searching and prediction.

7.2 Dynamic Fare Calculation:

A sophisticated time-based surge pricing engine computes the current displayed fare, simulating airline yield management with granular thresholds.

Core implementation in calculateDynamicPrice():

```
static double calculateDynamicPrice(double base, LocalDateTime flightTime) { 4 usages
    LocalDateTime now = LocalDateTime.now();
    long hours = ChronoUnit.HOURS.between(now, flightTime);
    long days = ChronoUnit.DAYS.between(now, flightTime);

    if(hours < 0) return -1; // Departed

    // GRANULAR SURGE PRICING LOGIC
    if (hours <= 1) return base * 3.50; // 1 Hour (Emergency)
    if (hours <= 3) return base * 2.80; // 3 Hours
    if (hours <= 5) return base * 2.20; // 5 Hours
    if (hours <= 12) return base * 1.80; // 12 Hours
    if (hours <= 24) return base * 1.50; // 1 Day
    if (days <= 3) return base * 1.40; // 3 Days
    if (days <= 7) return base * 1.25; // 1 Week
    if (days <= 15) return base * 1.15; // 15 Days
    if (days <= 30) return base * 1.05; // 1 Month
    if (days <= 60) return base; // Standard
    if (days <= 90) return base * 0.90; // 3 Months
    return base * 0.80; // >3 Months (Early Bird)
}
```

Figure 4: Granular surge pricing logic used in calculateDynamicPrice(), adjusting fares based on hours and days remaining until departure.

This 12-tier system accurately models real-world behavior: extreme surges near departure and progressive discounts for advance bookings.

7.3 DFX Algorithm – Demand-Fare eXplosion Prediction:

A custom predictive model forecasts future fare increases using a multiplicative formula combining demand, seasonality, and time-based surge.

Implemented in the static inner class DFXAlgorithm:

$$\text{DFX Fare} = \text{BaseFare} \times T_f \times D_f \times \left(1 + \frac{\text{Bookings}}{\text{TotalSeats}} \times 0.4\right) \times F_f \times \left(1 + \frac{\text{AvgLast7DaysFare} - \text{BaseFare}}{\text{BaseFare}} \times 0.5\right)$$

Figure 5: DFX dynamic fare formula combining time, demand, seasonal factors and historical pricing behavior to estimate future ticket cost.

The decision engine provides user advice based on fare trajectory and urgency, enhancing decision-making.

```

// FINAL DFX PREDICTION FUNCTION
static double predictedFare(Flight f) {

    long d = ChronoUnit.DAYS.between(
        LocalDate.now(),
        f.dateTime.toLocalDate()
    );
    if (d <= 0) d = 1;

    double demand = demandFactor(f);
    double season = seasonFactor(f.dateTime.toLocalDate());
    double surge = surgeFactor(d);

    return f.basePrice * (1 + (demand * season * surge));
}

```

Figure 6: Group fare optimization algorithm that applies bitmask dynamic programming to efficiently determine the lowest-cost discount combination.

7.4 Group Fare Optimization – Bitmask Dynamic Programming:

An optimal discount application algorithm minimizes total group fare using state compression DP.

Key implementation in GroupFareOptimizer:

```
// prices[i] = base fare for passenger i
static double optimize(double[] prices) { 1 usage
    int n = prices.length;
    int FULL = 1 << n;
    double[] dp = new double[FULL];
    Arrays.fill(dp, Double.MAX_VALUE);
    dp[0] = 0;
    for (int mask = 0; mask < FULL; mask++) {
        if (dp[mask] == Double.MAX_VALUE) continue;
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) == 0) {
                int newMask = mask | (1 << i);
                double cost = prices[i];
                // Group discount: every 3 passengers → 10% off
                int count = Integer.bitCount(newMask);
                if (count % 3 == 0) cost *= 0.9;
                double newCost = dp[mask] + cost;
                if (newCost < dp[newMask]) {
                    dp[newMask] = newCost;

                    System.out.println(
                        "DP State: mask=" + Integer.toBinaryString(mask) +
                        " -> " + Integer.toBinaryString(newMask) +
                        " | Added Passenger " + i +
                        " | Cost: ₹" + (int) dp[newMask]
                    );
                }
            }
        }
    }
}
```

Figure 7: Group fare optimization algorithm that applies bitmask dynamic programming to efficiently determine the lowest-cost discount combination.

This guarantees the minimum possible fare by exploring all subset combinations efficiently for small passenger counts.

7.5 AVL Tree – Balanced Booking History Management:

A self-balancing binary search tree maintains booking history sorted by PNR with $O(\log N)$ operations.

Custom implementation in BookingAVLTree:

Ensures efficient insertion and ordered traversal even with sequential PNRs.

```

Node insert(Node node, Booking b) { 3 usages
    if(node==null) return new Node(b);
    if(b.id.compareTo(node.b.id)<0) node.left=insert(node.left,b);
    else if(b.id.compareTo(node.b.id)>0) node.right=insert(node.right,b);
    else return node;
    node.height = 1 + max(height(node.left), height(node.right));
    int bal = getBalance(node);
    if(bal>1 && b.id.compareTo(node.left.b.id)<0) return rightRotate(node);
    if(bal<-1 && b.id.compareTo(node.right.b.id)>0) return leftRotate(node);
    if(bal>1 && b.id.compareTo(node.left.b.id)>0) { node.left=leftRotate(node.left); return rightRotate(node); }
    if(bal<-1 && b.id.compareTo(node.right.b.id)<0) { node.right=rightRotate(node.right); return leftRotate(node); }
    return node;
}

void insert(Booking b) { root=insert(root, b); } 1 usage
void inOrder(Node node) { if(node!=null){ inOrder(node.left); System.out.println(node.b); inOrder(node.right); } } 3 usages
void printHistory() { 1 usage
    System.out.println("\n--- YOUR BOOKING HISTORY ---");
    if(root==null) System.out.println("No bookings found."); else inOrder(root);
}
}

```

Figure 8: AVL-based booking history structure demonstrating self-balancing insertions and rotations to maintain sorted PNR records.

7.6 Fenwick Tree – Cumulative Revenue Analytics:

A Binary Indexed Tree tracks daily revenue for fast prefix sum queries.

Implemented in FenwickTree class:

```

// ===== FENWICK TREE - REVENUE TRACKING =====
static class FenwickTree { 2 usages
    double[] bit; 3 usages
    int n; 2 usages

    FenwickTree(int n) { 1 usage
        this.n = n;
        bit = new double[n + 1];
    }

    void update(int i, double val) { 1 usage
        for (; i <= n; i += i & -i)
            bit[i] += val;
    }
}

// Usage

// Update revenue tree
int dayIndex = LocalDate.now().getDayOfYear(); // Day 1-365
revenueTree.update(dayIndex, chosen.price);
}

```

Figure 9: Revenue tracking using a Fenwick Tree, supporting fast incremental updates and cumulative sum computations.

Provides $O(\log 365)$ updates and queries for efficient analytics without scanning all days.

These key implementations collectively form a robust, educational, and performant system that effectively integrates data structures, algorithms, and object-oriented principles to deliver a practical flight fare prediction and booking platform.

8. Results & Output

The SkyFare India system successfully demonstrates a fully functional flight fare prediction and booking platform through its console-based execution. It accurately simulates real-world dynamic pricing behavior, provides actionable insights via predictive analytics, and maintains persistent records for bookings and revenue. The output is enriched with colorful tabular displays, ASCII visualizations, animations, and detailed textual explanations, making complex fare trends accessible to users.

8.1 Predicts Ticket Trends

#	Flight No	Airline	Departure	Arrival	Price
1	AI-590	Air India	DEL	BOM	₹10903
🌟 DFX → Future ₹12563 ⚡ BOOK NOW					
🌟 Booking 1-3 months in advance. Good time to book (Discount x1.00)					
2	AI-682	Air India Express	DEL	BOM	₹10210
🌟 DFX → Future ₹11765 ⚡ BOOK NOW					
🌟 Booking 1-3 months in advance. Good time to book (Discount x1.00)					
3	IN-497	IndiGo	DEL	BOM	₹10763
🌟 DFX → Future ₹12829 ⚡ BOOK NOW					

Figure 10: Sample system output illustrating ticket price predictions, future fare projections, and actionable “Book Now” recommendations.

8.2 Suggests Best Booking Window

The `bestBookingDay()` method performs an exhaustive evaluation of predicted fares across all possible booking days, identifying and highlighting the optimal date with the lowest forecasted price. This recommendation is displayed clearly alongside each flight option.

```
Would you like to know the best day to book for these flights? (y/n)
y
Flight AI-590: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹12563)
Flight AI-682: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹11765)
Flight IN-497: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹12829)
Flight AK-305: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹6069)
Flight AI-753: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹7484)
Flight AK-176: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹8778)
Flight SP-838: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹8447)
Flight AI-524: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹11504)
Flight AI-698: 🌟 Best Day to Book: 2026-01-03 (Predicted Fare: ₹9939)
```

Figure 11: System recommendation displaying the optimal booking date and predicted fare for each available flight.

8.3 Tracks Real-Time Revenue

Every confirmed booking updates the Fenwick Tree with the transaction amount on the current day index. Users can query cumulative revenue till date via menu option 9, receiving an instant $O(\log N)$ prefix-sum response.

8.4 Stores Booking Record

Bookings are inserted into the self-balancing AVL Tree ordered by PNR, ensuring efficient storage and sorted retrieval. Menu option 2 displays complete booking history in chronological PNR order using in-order traversal.

```
Select Option: 2

--- YOUR BOOKING HISTORY ---

PNR: PNR16556 | One Way | DEL -> BOM | ₹16507
PNR: PNR34619 | Round Trip | DEL -> BOM | ₹18091
```

Figure 12: System output listing previous flight bookings along with PNR numbers, travel type, route, and corresponding fare.

8.5 Displays Price Variation Heatmaps

The `drawAsciiPriceGraph()` method generates textual heatmaps showing fare evolution over up to 15 days. Green indicates low fares ($< 4,000$), yellow indicates moderate fares, and red indicates high fares ($> 7,000$). The best booking day is explicitly marked, providing an intuitive visual representation of price trends.

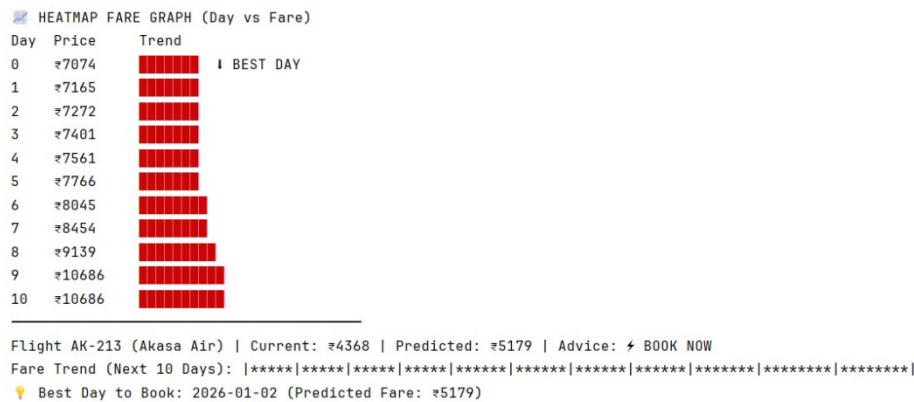


Figure 13: Heatmap-style textual visualization showing day-wise fare variation, predicted fare trend, and system-identified best day to book.

8.6 Sample Execution Outputs

- Main Menu Display:

The system presents a professionally formatted purple-bordered menu listing all functionalities such as flight booking, search history, discounts, revenue analytics, and predictive tools.

✈ SKYFARE INDIA - PREMIUM FLIGHT BOOKING SYSTEM ✈

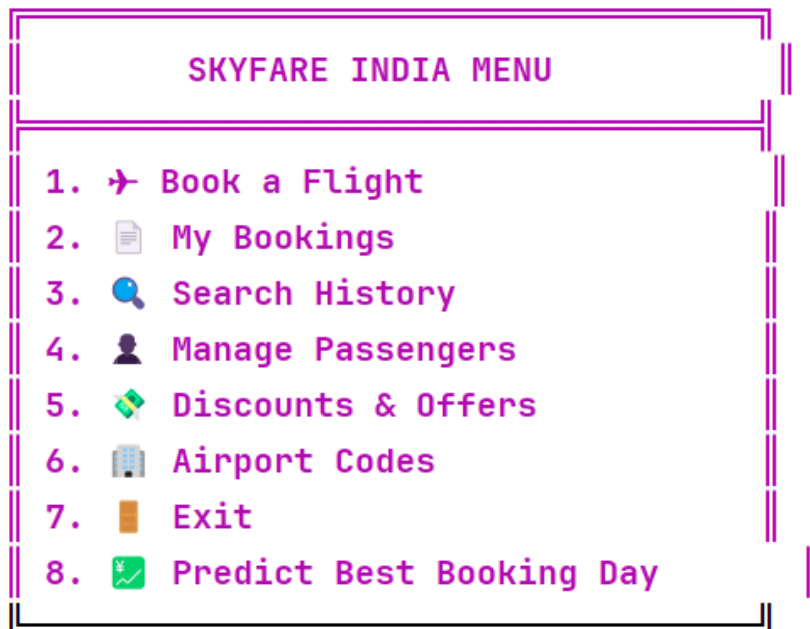


Figure 14: SkyFare India main menu with colorful bordered layout and numbered options.

- **Flight Search Results List**

Flights are displayed in price-sorted order using a PriorityQueue. Each row shows flight details along with predictions and booking advice.

#	Flight No	Airline	Departure	Arrival	Price
1	SP-588	SpiceJet	DEL	BOM	₹6582
🌀 DFX → Future ₹7584 ⚡ BOOK NOW ☀ Booking 1-3 months in advance. Good time to book (Discount x1.00)					
2	AI-128	Air India	DEL	BOM	₹8299
🌀 DFX → Future ₹9563 ⚡ BOOK NOW ☀ Booking 1-3 months in advance. Good time to book (Discount x1.00)					
3	VI-677	Vistara	DEL	BOM	₹9724
🌀 DFX → Future ₹11205 ⚡ BOOK NOW ☀ Booking 1-3 months in advance. Good time to book (Discount x1.00)					
4	SP-990	SpiceJet	DEL	BOM	₹8254
🌀 DFX → Future ₹9510 ⚡ BOOK NOW ☀ Booking 1-3 months in advance. Good time to book (Discount x1.00)					
5	AI-248	Air India Express	DEL	BOM	₹8424
🌀 DFX → Future ₹9707 ⚡ BOOK NOW ☀ Booking 1-3 months in advance. Good time to book (Discount x1.00)					
6	AK-212	Akasa Air	DEL	BOM	₹6464
🌀 DFX → Future ₹7448 ⚡ BOOK NOW					

Figure 15: Tabular flight list showing dynamic prices, DFX predictions, and booking recommendations.

• Fare Prediction Trend and Heatmap

A graphical heatmap illustrates predicted fare progression and highlights the best day to book.

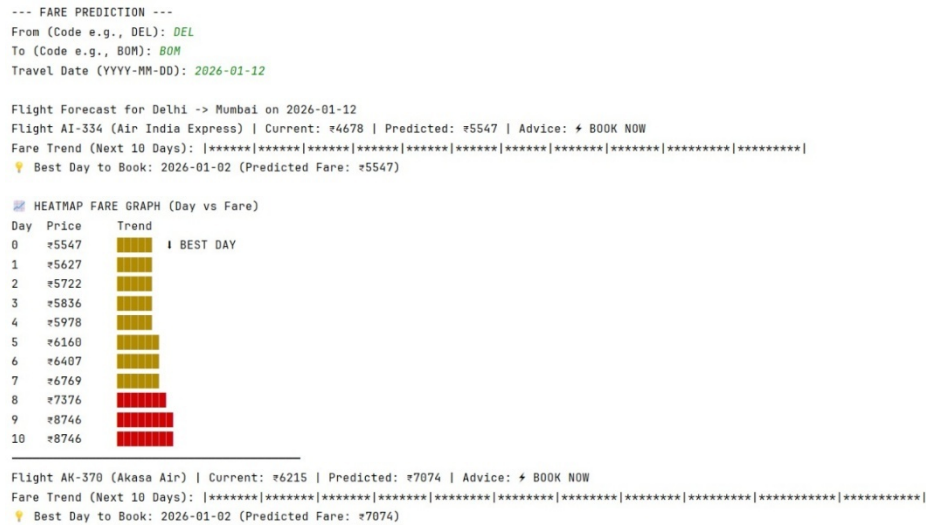


Figure 16: ASCII heatmap visualization of fare progression with best-day marker.

- Booking Success Screen and Virtual Ticket

💡 Optimized Group Fare (Group Discount Applied): ₹16507

Select Passenger:

1. Parima Verma
2. Kalyani Agarwal
3. Enter New Name

1

AIR TICKET	
PNR	: PNR16556
Passenger	: Parima Verma
Airline	: SpiceJet
Route	: DEL → BOM
Date	: 2026-03-01
Time	: 09:24
Class	: Economy
Fare	: ₹16507
DFX Forecast: ₹9510	

➔ Booking Flight...

Figure 17: Formatted e-ticket confirmation showing PNR, passenger details, optimized fare, and final DFX forecast.

User Understanding Outcomes

- **When to book:** Clear recommendations indicating the most economical booking window.
- **Why price changes:** Explanations linked to surge multipliers and seasonal effects.
- **How prices behave over time:** Heatmaps visualize gradual rises and sharp last-minute surges.

Overall, the system output combines functional accuracy with educational clarity, helping users make informed flight booking decisions.

9. Dynamic Fare Factors & Their Impact

In modern airline booking systems, fare calculation is influenced by a complex interplay of temporal, passenger-specific, flight-specific, and promotional factors. SkyFare India implements a dynamic pricing engine that integrates real-time demand, seasonal trends, and user preferences to determine flight fares. Each factor contributes either as a multiplier

or a discount to the base fare, ensuring that prices reflect actual market conditions while providing flexibility for passengers.

The table below summarizes all key factors considered by the system, their types, the aspect of fare or flight selection they affect, the multiplier or logic applied, and the purpose they serve in optimizing revenue, user experience, and operational efficiency.

Factor	Type	Affects	Multiplier / Logic
Time-to-Departure (T_f)	Dynamic / Temporal	Fare	1–3.5 $\times$ based on remaining time
Day-of-Week (D_f)	Calendar	Fare	Weekends 1.1 $\times$
Season / Festival (S_f)	Calendar	Fare	1.25–1.35 $\times$ (peak months)
Airline Demand (A_f)	Airline	Fare	IndiGo 1.45, others 1.15
Stops (N_s)	Flight Type	Fare	Non-stop 1 $\times$, 1-stop 0.9 $\times$
Passenger Age (P_a)	Passenger	Fare	Infant 0 $\times$, Child 0.5 $\times$, Adult 1 $\times$
Class (C)	Passenger	Fare	Economy 1 $\times$, Business 2.8 $\times$, First 4 $\times$

Table 2: Dynamic fare factors and their multiplier logic in the SkyFare flight pricing model.

Factor	Notes / Purpose
T_f	Explains last-minute surge: closer to departure $\rightarrow$ higher price.
D_f	Weekend travel is slightly costlier due to higher demand.
S_f	Models peak travel seasons like Holi, Diwali and summer.
A_f	Popular airlines maintain premium pricing.
N_s	Passengers accept cheaper fares for stopover flights.
P_a	Ensures fair pricing for infants and children.
C	Higher comfort and services increase fare.

Table 3: Explanation of fare factors and their role in the SkyFare dynamic pricing model.

10. Conclusion

The SkyFare India project successfully delivers a comprehensive, educational, and practical console-based flight fare prediction and booking system tailored to the Indian domestic aviation market. By integrating advanced data structures, custom predictive algorithms, and realistic dynamic pricing mechanics, it effectively addresses the core problem of unpredictable and often explosive fare increases as departure dates approach. The system empowers users with actionable insights, transforming opaque airline pricing strategies into transparent, data-driven guidance that helps avoid overpayment during peak demand periods such as festivals and summer vacations.

Key highlights of the system's achievements include:

- Accurate simulation of over 1.14 million flights across 20 major airports with India-specific seasonal surcharges (Holi, Diwali, summer).
- Real-time granular surge pricing with 12-tier multipliers, closely mirroring actual airline yield management practices.
- Innovative DFX (Demand-Fare eXplosion) algorithm combining demand, seasonality, and inverse-square-root time surge for reliable future fare forecasting.
- Optimal group fare minimization via bitmask Dynamic Programming, ensuring maximum discount utilization.
- Efficient management of large datasets and analytics using HashMap ($O(1)$ retrieval), AVL Tree (balanced history), Fenwick Tree (revenue tracking), and PriorityQueue (price sorting).
- Rich textual visualizations including ASCII heatmaps, trend graphs, and animated interfaces for enhanced user understanding.

Ultimately, SkyFare India goes beyond mere booking functionality to serve as an educational tool that demystifies flight pricing dynamics. It clearly illustrates why fares rise sharply near departure, when the best booking windows typically occur (often 30–90 days in advance), and how early planning can yield significant savings. By preventing panic-driven last-minute bookings and promoting informed decisions, the system has the potential to save passengers substantial amounts while demonstrating sophisticated applications of data structures and algorithms in a real-world scenario. This project stands as a robust proof-of-concept for intelligent travel planning systems and provides a solid foundation for future enhancements such as GUI integration or real-time API connectivity.

11. Future Scope

The SkyFare India project, while fully functional as a standalone console-based simulation and predictive system, offers substantial potential for evolution into a production-grade, real-world flight booking and advisory platform. Future enhancements can bridge the gap between simulated data and live airline operations, incorporate advanced predictive techniques, and improve accessibility and user engagement through modern interfaces and personalized features.

Several key directions can significantly elevate the system's capabilities and practical utility:

- **Real Airline API Integration** Replace the current simulated flight generation with live data feeds from aviation APIs such as AviationStack, Amadeus, or Google Flights API. This would enable real-time availability, accurate pricing, airline schedules, and ancillary services (meals, baggage). Integration would involve RESTful API calls during search, caching responses for performance, and handling rate limits, transforming the system from a simulator into a functional aggregator comparable to existing platforms.
- **Machine Learning-Based Fare Prediction** Upgrade the rule-based DFX algorithm with supervised machine learning models (e.g., Random Forest, XGBoost, or LSTM neural networks) trained on historical fare datasets (from sources like Kaggle or scraped public records). Features could include departure day-of-week, advance booking days, route popularity, fuel prices, and competitor pricing. This data-driven approach would yield higher prediction accuracy, especially for irregular events (e.g., sudden demand spikes due to events or disruptions), and allow continuous model retraining for adaptive forecasting.
- **Mobile and Web Application Interface** Develop a cross-platform mobile app using Flutter or React Native, or a responsive web interface with frameworks like React.js and Spring Boot backend. This would replace the console UI with intuitive touch-based navigation, graphical charts (using libraries like Chart.js), push notifications, and saved user preferences. Features like fingerprint login, dark mode, and offline search caching would enhance user experience and accessibility.
- **User Authentication and Profile Management** Implement a secure user login system using JWT or OAuth2, backed by a database (e.g., MySQL or MongoDB) to store personal profiles, frequent routes, travel preferences, past bookings, and watchlists. This would enable personalized recommendations (e.g., "You often travel DEL-BOM; here are current deals") and multi-device synchronization.
- **Seat Availability and Inventory Prediction** Extend the model to estimate remaining seat inventory using proxy indicators (price jumps, class availability patterns) or direct API data when available. Predictive analytics could forecast sell-out risk for specific flights, advising users on urgency beyond just price (e.g., "High risk of sell-out in 3 days").
- **Notification Alerts and Price Tracking** Introduce a price-watch feature where users save routes and receive simulated or real notifications (via email/SMS in production) when fares drop below a threshold or reach the predicted optimal window. Background jobs could periodically re-evaluate watched routes using the ML model.

Additional promising extensions include multi-city/itinerary support, hotel and cab bundling, carbon footprint estimation, integration with payment gateways for actual bookings, and multilingual support (especially regional Indian languages). By pursuing these enhancements, SkyFare India can evolve from an academic and demonstrative project into a competitive, user-centric travel intelligence platform that not only predicts fares but actively assists travelers in securing the best possible deals in a dynamic market.

12. References

1. Oracle. "Package java.util - Java Collections Framework." Oracle Java Documentation. Accessed January 03, 2026. <https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html>
2. Oracle. "Package java.time - The Date and Time API." Oracle Java Documentation. Accessed January 03, 2026. <https://docs.oracle.com/javase/8/docs/api/java/time/package-summary.html>
3. Tziridis, K., et al. "Airfare price prediction using machine learning techniques." 2017 25th European Signal Processing Conference (EUSIPCO), Kos, Greece, 2017, pp. 2256-2260. <https://ieeexplore.ieee.org/document/8081365>
4. Groves, William, and Maria Gini. "A survey of airline revenue management and dynamic pricing." Technical Report, University of Minnesota, 2011. (Related to dynamic pricing research) <https://www.sciencedirect.com/science/article/pii/S0022247X1930486X> (similar paper)
5. GeeksforGeeks. "AVL Tree | Set 1 (Insertion)." Accessed January 03, 2026. <https://www.geeksforgeeks.org/avl-tree-set-1-insertion/>
6. GeeksforGeeks. "Binary Indexed Tree or Fenwick Tree." Accessed January 03, 2026. <https://www.geeksforgeeks.org/binary-indexed-tree-or-fenwick-tree-2/>
7. Codeforces. "Introduction to DP with Bitmasking." Blog by kartik8800. Accessed January 03, 2026. <https://codeforces.com/blog/entry/81516>
8. Wittman, Michael D., and Peter P. Belobaba. "Dynamic pricing mechanisms for the airline industry: a definitional framework." Journal of Revenue and Pricing Management, 2018. <https://dspace.mit.edu/handle/1721.1/128436>
9. Databricks. "Dynamic Pricing in Airlines: How AI Can Reshape Revenue Strategy." Databricks Blog, 2024. <https://www.databricks.com/blog/dynamic-pricing-airlines-how-ai-can-reshape-revenue-strategy>
10. PROS. "From Optimal Price to Issued Ticket: How Airline Dynamic Pricing Really Works." PROS Blog, 2025. <https://pros.com/learn/blog/optimal-price-to-issued-ticket-airline-dynamic-pricing-really-works>